

下面我先把这次大作业的核心要求翻译成“我们到底要做什么”，再给出详细任务列表、报告模板目录和三人分工方案。整体目标不是做一个功能特别多的知乎，而是做一个**能体现微服务、Redis、MQ、网关限流、AI流式响应、生产环境架构思考**的后端系统。

一、题目核心理解

本题是 **基于 AI 的智能知识库与内容发布平台**，可以理解成一个“带 AI 能力的知乎后台系统”。

它的重点不是前端，也不是堆很多普通 CRUD，而是要证明我们掌握了：

1. **微服务拆分**
2. **网关统一入口与动态限流**
3. **Redis 热榜**
4. **消息队列异步处理**
5. **AI 大模型集成**
6. **SSE / Streaming 流式返回**
7. **生产环境高并发、高可用设计**

所以代码上要做到“本地能跑通”，报告上要做到“架构思想讲清楚”。

二、建议最终项目定位

项目名称可以定为：

AI-KnowledgeHub：基于 AI 的智能知识库与内容发布平台

系统包含以下核心业务：

用户注册登录后，可以创建文章草稿、发布文章、查看文章详情、评论、点赞。用户阅读或点赞文章后，文章热度会实时更新到 Redis ZSET 热榜中。用户发布长文后，文章事件会进入 MQ，由多个消费者异步处理，例如 AI 标签提取服务、AI 合规检测服务。用户写作时，可以调用 AI 续写接口，后端通过 SSE 流式返回 AI 生成内容。

三、推荐技术栈

为了降低开发难度，同时满足验收要求，建议采用下面这套。

1. 后端框架

推荐：

Spring Boot + Spring Cloud Gateway

理由：

- 和 Web 后端课程匹配度高。
- Swagger/OpenAPI 支持成熟。
- Redis、RabbitMQ、MySQL 集成简单。
- 网关限流、JWT 鉴权、微服务拆分好写。
- 老师验收时比较容易理解。

如果你们更熟 Python，也可以用：

FastAPI + Nginx/API Gateway + Redis + RabbitMQ

但由于题目明确要求网关、微服务、中间件，Spring Boot 会更像一个完整课程项目。

2. 中间件

类型	推荐选择	用途
关系型数据库	MySQL	存用户、文章、评论、点赞、AI处理结果
Redis	Redis ZSET + 缓存 + 限流计数	热榜、文章详情缓存、限流
MQ	RabbitMQ	文章发布后的异步广播处理
网关	Spring Cloud Gateway	统一入口、JWT校验、动态限流
API文档	Knife4j / Swagger OpenAPI	自动生成接口文档
AI接口	OpenAI兼容接口 / 通义 / 智谱 / Mock LLM	AI续写、标签提取、合规检测
测试工具	Postman / Apifox	验收流程导入测试

四、微服务拆分方案

验收要求至少 3 个独立服务。建议拆成 5 个，既清晰又不算太复杂。

服务 1: gateway-service 网关服务

负责:

- 所有请求统一入口
- JWT Token 校验
- 普通用户 / 管理员权限判断
- IP 限流
- 从配置中心或数据库/Redis 动态读取限流配置
- 路由转发到后端服务

重点体现:

- 网关层限流
 - 动态限流参数
 - 统一鉴权
-

服务 2: user-service 用户服务

负责:

- 用户注册
- 用户登录
- JWT Token 签发
- 用户注销
- 用户角色管理
- 管理员权限识别

核心接口:

```
POST /api/user/register
POST /api/user/login
POST /api/user/logout
GET /api/user/profile
```

服务 3: article-service 文章服务

负责:

- 创建文章草稿
- 修改文章
- 发布文章
- 逻辑删除文章
- 分页获取最新文章
- 获取文章详情
- 点赞文章
- 评论文章
- 阅读文章时更新热度
- 发布文章后发送 MQ 消息

核心接口：

```
POST    /api/articles/draft
PUT      /api/articles/{id}
POST     /api/articles/{id}/publish
DELETE   /api/articles/{id}
GET      /api/articles/latest
GET      /api/articles/{id}
POST     /api/articles/{id}/like
POST     /api/articles/{id}/comments
```

服务 4：ranking-service 热榜服务

负责：

- 使用 Redis ZSET 维护文章热度
- 阅读文章增加热度
- 点赞文章增加热度
- 获取全站 Top 10 热榜
- 定时衰减热度，避免老文章长期霸榜

核心接口：

```
POST /api/ranking/articles/{id}/view
POST /api/ranking/articles/{id}/like
GET  /api/ranking/top10
```

也可以把阅读和点赞热度更新放到 article-service 中，ranking-service 专门提供 Top10 查询。但报告里建议把它作为独立服务讲，会更体现微服务拆分。

服务 5：ai-service AI 服务

负责：

- AI 续写流式接口
- 监听 MQ 消息
- 文章发布后异步提取标签
- 文章发布后异步进行合规检测
- 保存 AI 处理结果

核心接口：

```
GET  /api/ai/continue-writing/stream
POST /api/ai/continue-writing
GET  /api/ai/articles/{id}/analysis
```

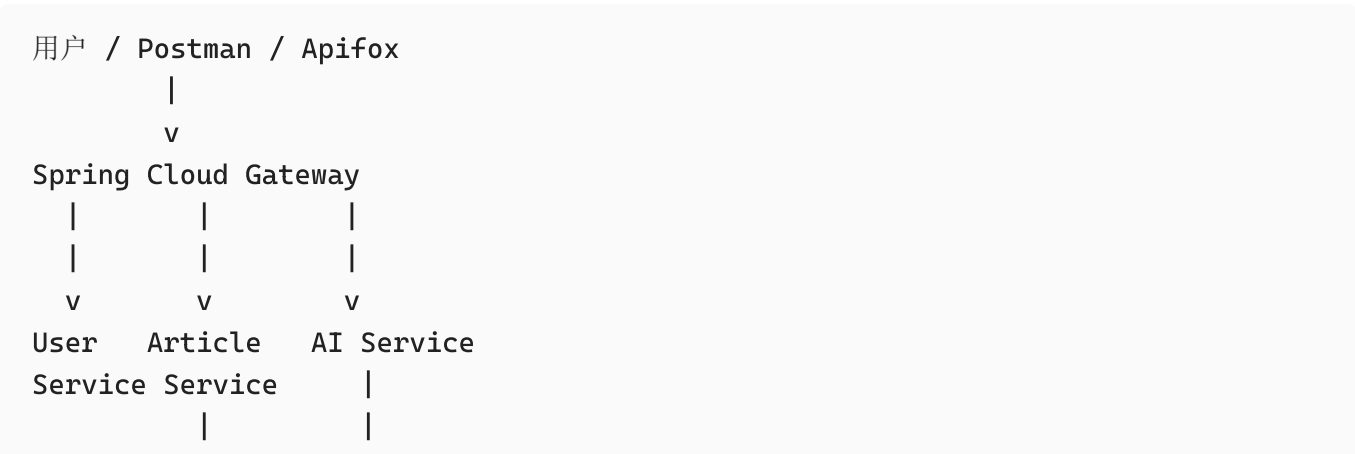
其中 GET /api/ai/continue-writing/stream 使用 SSE：

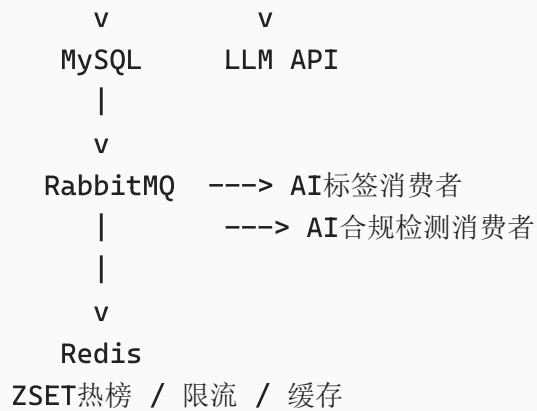
```
Content-Type: text/event-stream
```

逐字或逐句返回 AI 续写结果。

五、系统整体架构图建议

报告里可以画成这样：





六、详细任务列表

下面是按开发阶段拆出来的任务列表，你们可以直接作为项目 Todo 使用。

阶段 0：项目初始化

0.1 创建代码仓库

任务内容：

- 创建 Git 仓库。
- 建立统一项目目录。
- 添加 README。
- 添加 `.gitignore`。
- 确定代码规范。

建议目录：

```
ai-knowledgehub/
├─ gateway-service/
├─ user-service/
├─ article-service/
├─ ranking-service/
├─ ai-service/
├─ common/
├─ docs/
└─ postman/
```

```
├─ docker-compose.yml
└─ README.md
```

0.2 编写 Docker Compose

需要启动：

- MySQL
- Redis
- RabbitMQ
- Nacos 可选
- 各个微服务可本地启动，也可以后续容器化

最低要求：

```
MySQL + Redis + RabbitMQ
```

0.3 统一接口规范

包括：

- 返回格式统一
- 错误码统一
- JWT Header 规范
- 分页参数规范
- 时间格式规范

统一返回示例：

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

阶段 1：基础支撑功能开发

1.1 用户注册

实现内容：

- 用户名唯一校验
- 密码加密存储
- 默认角色为普通用户
- 支持管理员角色初始化

数据库表：

```
user
```

1.2 用户登录

实现内容：

- 校验用户名密码
 - 登录成功生成 JWT Token
 - 返回用户基本信息和角色
-

1.3 JWT 鉴权

实现内容：

- 网关解析 Token
 - 校验 Token 是否有效
 - 将用户 ID、角色传给后端服务
 - 对管理员接口做权限拦截
-

1.4 注销登录

实现内容：

- 简单版：前端丢弃 Token
- 加分版：Redis 维护 Token 黑名单

报告里可以写：

本系统在本地原型中采用 JWT 无状态认证，并使用 Redis Token 黑名单实现注销后的 Token 失效控制。

阶段 2：文章内容管理

2.1 创建文章草稿

实现内容：

- 登录用户创建草稿
- 文章状态为 DRAFT
- 保存标题、正文、作者 ID

2.2 修改文章

实现内容：

- 作者本人可以修改
- 管理员可以修改
- 已删除文章不能修改

2.3 发布文章

实现内容：

- 状态从 DRAFT 改为 PUBLISHED
- 设置发布时间
- 发送 MQ 消息：ArticlePublishedEvent

MQ 消息示例：

```
{
  "articleId": 1001,
  "title": "Redis ZSET实现热榜",
  "authorId": 12,
  "content": "文章正文..."
```

```
"publishTime": "2026-06-02 20:00:00"
}
```

2.4 逻辑删除文章

实现内容：

- 修改 `deleted` 字段
- 不物理删除数据库记录

2.5 分页获取最新文章

实现内容：

- 只返回已发布、未删除文章
- 按发布时间倒序
- 支持分页

2.6 获取文章详情

实现内容：

- 查询文章详情
- 阅读量增加
- Redis 热度增加
- 可选：文章详情缓存

建议：

```
阅读一次：热度 +1
点赞一次：热度 +5
评论一次：热度 +3
```

阶段 3：互动基础功能

3.1 评论入库

实现内容：

- 用户对文章发表评论
 - 评论保存到 MySQL
 - 支持查看文章评论列表
 - 评论后热度增加
-

3.2 点赞功能

实现内容：

- 用户点赞文章
- 防止同一用户重复点赞
- MySQL 保存点赞记录
- 文章点赞数增加
- Redis ZSET 热度增加

推荐做法：

```
article_like 表中设置 user_id + article_id 唯一索引
```

阶段 4：核心架构功能开发

4.1 网关动态限流

这是高分重点之一。

实现目标

限制某 IP 对文章详情接口的访问频率，例如：

```
同一 IP 每 10 秒最多访问文章详情接口 20 次
```

超过限制返回：

```
{  
  "code": 429,  
  "message": "访问过于频繁，请稍后再试"  
}
```

推荐实现方式

在 Gateway 中使用 Redis 计数。

Key 设计：

```
rate_limit:{ip}:{path}
```

示例：

```
rate_limit:127.0.0.1:/api/articles/1001
```

设置过期时间：

```
10 秒
```

动态配置来源

简单且好实现的方案：

```
Redis Hash 保存限流配置
```

例如：

```
rate_limit_config:article_detail
```

字段：

```
windowSeconds = 10  
maxRequests = 20  
enabled = true
```

管理员接口可以动态修改配置：

```
PUT /api/admin/rate-limit/article-detail
```

报告中说明：

网关每次处理请求时从 Redis 读取当前限流配置，配置修改后无需重启服务即可生效。

4.2 Redis ZSET 实时热榜

这是题目明确要求的核心功能。

数据结构

使用 Redis ZSET：

```
article:hot:ranking
```

其中：

```
member = articleId  
score = hotScore
```

热度规则

建议设计：

```
阅读文章: +1  
点赞文章: +5  
评论文章: +3  
发布文章: +2
```

获取 Top10

Redis 命令：

```
ZREVRANGE article:hot:ranking 0 9 WITHSCORES
```

然后根据 articleId 批量查询文章标题、作者、摘要等信息。

为什么用 ZSET

报告里可以写：

文章热榜需要频繁更新分数，并且需要快速获取分数最高的 Top N 文章。Redis ZSET 内部按 score 维护有序集合，适合实现排行榜类场景。相比每次从 MySQL 中 order by 统计热度，ZSET 可以减少数据库压力，并且 Top10 查询效率更高。

4.3 长文异步广播处理

这是 MQ 的核心体现。

业务场景

用户发布长文后，不能让发布接口一直等待 AI 处理完成。正确做法是：

发布文章成功 -> 发送 MQ 消息 -> 立即返回发布成功

然后下游消费者异步处理：

AI 标签服务消费者
AI 合规检测消费者

RabbitMQ 模式

建议使用：

Fanout Exchange

原因：

- 一篇文章发布后，需要多个下游服务都收到同一条消息。

- 标签提取和合规检测互不影响。
- 某个消费者失败时，不影响其他消费者。

MQ 结构



消费者 A：AI 标签提取

输入：

文章标题 + 正文

输出：

标签列表，例如 **Redis**、后端开发、微服务

保存到：

`article_ai_tag`

消费者 B：AI 合规检测

输入：

文章正文

输出：

PASS / REJECT / REVIEW

保存到：

article_audit_result

失败处理

本地原型可以简单做：

- 消费失败打印错误日志
- 手动重试

报告里的生产方案可以写：

- 消息确认机制 ACK
- 消费失败进入重试队列
- 多次失败进入死信队列
- 后台人工处理异常文章

4.4 AI 流式创作辅助

这是 AI + SSE 的核心体现。

实现目标

用户输入一段文字：

```
{  
  "prompt": "请继续写一段关于Redis热榜设计的内容"  
}
```

后端不是一次性返回完整结果，而是逐字或逐句返回：

data: Redis ZSET 很适合实现排行榜场景...

data: 因为它可以根据 score 进行排序...

data: 在高并发系统中...

接口设计

```
GET /api/ai/continue-writing/stream?prompt=xxx
```

响应头:

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

本地演示方式

可以用:

- 浏览器直接访问 SSE 地址
- Postman 测试流式响应
- 简单 HTML 页面测试

生产环境说明

报告里要重点写:

LLM 调用响应慢、费用高、并发受限, 因此不能把所有 AI 请求都直接打到第三方 API。生产环境中可以使用请求排队、限流、结果缓存、相似 Prompt 缓存、异步任务队列和降级策略来优化 AI 调用链路。

七、数据库表设计建议

1. user 用户表

```
id
username
password_hash
role
status
created_at
updated_at
```

2. article 文章表

```
id
author_id
title
content
summary
status
view_count
like_count
comment_count
deleted
created_at
updated_at
published_at
```

3. comment 评论表

```
id
article_id
user_id
content
deleted
created_at
```

4. article_like 点赞表

```
id
article_id
user_id
created_at
```

唯一约束：

```
article_id + user_id
```

5. article_ai_tag AI 标签表

```
id
article_id
tags
model_name
created_at
```

6. article_audit_result AI 合规检测表

```
id
article_id
result
reason
model_name
created_at
```

7. rate_limit_rule 限流规则表，可选

如果你们想从 MySQL 管理规则，可以加这个表：

```
id
rule_name
path_pattern
window_seconds
max_requests
```

```
enabled
created_at
updated_at
```

不过从实现难度看，直接放 Redis 配置更简单。

八、核心业务链路设计

链路 1：用户登录链路

用户登录

- > `user-service` 校验用户名密码
- > 生成 JWT Token
- > 返回 Token
- > 后续请求携带 `Authorization Header`
- > `gateway-service` 校验 Token
- > 转发到对应微服务

链路 2：文章详情访问链路

用户请求文章详情

- > `gateway-service` 执行 IP 限流
- > 限流通过
- > `article-service` 查询文章详情
- > 更新阅读量
- > Redis ZSET 热度 +1
- > 返回文章详情

链路 3：文章点赞链路

用户点赞文章

- > `gateway-service` 校验 JWT
- > `article-service` 判断是否重复点赞
- > 写入 `article_like`
- > `article.like_count` +1

- > Redis ZSET 热度 +5
- > 返回点赞成功

链路 4：文章发布异步处理链路

用户发布文章

- > `article-service` 修改文章状态为 `PUBLISHED`
- > 发送 `ArticlePublishedEvent` 到 `RabbitMQ Fanout Exchange`
- > 立即返回发布成功
- > AI标签消费者收到消息
- > 调用 `LLM` 提取标签
- > 保存标签结果
- > AI合规消费者收到消息
- > 调用 `LLM` 做合规检测
- > 保存检测结果

链路 5：AI 流式续写链路

用户调用 AI 续写接口

- > `gateway-service` 转发请求
- > `ai-service` 调用 `LLM Streaming API`
- > `ai-service` 将模型输出转换为 `SSE`
- > 客户端逐字/逐句接收内容

九、实验报告模板

详见共享文档

十、三人任务分工方案

下面按三个人来分工。为了方便，我先命名为：

- 成员 A：后端主架构与用户/网关方向
- 成员 B：文章业务与 `Redis` 热榜方向
- 成员 C：AI、MQ、测试文档方向

你们后续可以把 A/B/C 替换成真实姓名。

成员 A：网关、用户认证、项目架构负责人

主要负责内容

- 1. 项目整体骨架搭建
- 2. gateway-service 开发
- 3. user-service 开发
- 4. JWT 鉴权
- 5. 网关动态限流
- 6. Docker Compose 基础环境
- 7. Swagger/OpenAPI 基础配置

详细任务

A-1 项目初始化

负责：

- 创建项目仓库。
- 搭建多模块工程。
- 配置公共依赖。
- 配置统一返回结果。
- 配置统一异常处理。
- 配置 Swagger/OpenAPI。

交付物：

完整项目骨架

README 初版

common 公共模块

A-2 user-service

负责接口：

```
POST /api/user/register
POST /api/user/login
POST /api/user/logout
GET  /api/user/profile
```

实现内容：

- 用户注册
 - 密码加密
 - 登录校验
 - JWT Token 生成
 - 用户角色字段
 - 用户信息查询
-

A-3 gateway-service

负责：

- 路由转发
- JWT 校验
- 管理员接口权限判断
- 请求头透传用户 ID 和角色

示例：

```
X-User-Id
X-User-Role
```

A-4 动态限流

负责：

- 文章详情接口限流
- Redis 限流计数
- 限流配置动态读取
- 管理员修改限流配置接口

建议实现：

A-5 报告负责章节

成员 A 负责报告中的：

- 1. 项目概述
- 2. 系统总体设计
- 5.1 用户认证与 JWT 鉴权
- 5.4 网关动态限流功能
- 8.1 为什么使用微服务架构
- 8.4 为什么使用网关限流
- 9.1 高可用设计

成员 B：文章业务、评论点赞、Redis 热榜负责人

主要负责内容

- 1. article-service 开发
- 2. 文章 CRUD
- 3. 评论功能
- 4. 点赞功能
- 5. Redis ZSET 热榜
- 6. 数据库 ER 图设计
- 7. 文章相关接口测试

详细任务

B-1 数据库设计

负责设计：

```
user
article
comment
article_like
```



```
article_ai_tag
article_audit_result
```

重点：

- 表结构完整。
 - 主外键关系清楚。
 - 点赞表有唯一索引。
 - 文章表支持草稿、发布、逻辑删除。
-

B-2 article-service

负责接口：

```
POST    /api/articles/draft
PUT      /api/articles/{id}
POST     /api/articles/{id}/publish
DELETE   /api/articles/{id}
GET      /api/articles/latest
GET      /api/articles/{id}
```

实现内容：

- 创建草稿
 - 修改文章
 - 发布文章
 - 逻辑删除
 - 分页查询
 - 文章详情
-

B-3 评论功能

负责接口：

```
POST /api/articles/{id}/comments
GET  /api/articles/{id}/comments
```

实现内容：

- 评论入库
 - 查询文章评论
 - 评论数更新
 - 评论后热度增加
-

B-4 点赞功能

负责接口：

```
POST /api/articles/{id}/like
```

实现内容：

- 防止重复点赞
 - 点赞记录入库
 - 文章点赞数更新
 - Redis 热度增加
-

B-5 ranking-service

负责接口：

```
GET /api/ranking/top10
```

实现内容：

- Redis ZSET 存储文章热度
 - 阅读、点赞、评论更新热度
 - 获取 Top10 文章
 - 根据 articleId 补充文章标题、作者、摘要
-

B-6 报告负责章节

成员 B 负责报告中的：

- 2. 需求分析
- 3. 数据库设计
- 5.2 文章管理功能
- 5.3 评论与点赞功能
- 5.5 Redis ZSET 热榜功能
- 8.2 为什么使用 Redis ZSET 实现热榜
- 9.3 数据库优化方案
- 9.4 Redis 生产环境方案

成员 C：AI、MQ、测试、演示视频与文档整合负责人

主要负责内容

- 1. ai-service 开发
- 2. RabbitMQ Fanout 广播
- 3. AI 标签提取消费者
- 4. AI 合规检测消费者
- 5. AI 流式续写 SSE
- 6. Postman / Apifox Collection
- 7. 演示视频脚本
- 8. 报告最终整合

详细任务

C-1 RabbitMQ 配置

负责：

- 创建 Fanout Exchange
- 创建两个队列
- 绑定队列到交换机

结构：

```
article.publish.exchange
  -> article.tag.queue
  -> article.audit.queue
```

C-2 文章发布事件监听

负责：

- 监听文章发布消息
- 解析 articleId、title、content
- 打印消费日志
- 保存处理结果

C-3 AI 标签提取

负责：

- 调用 LLM API 或 Mock LLM
- 根据文章内容生成标签
- 将标签保存到数据库

如果真实 API 不稳定，可以准备 Mock 模式：

根据关键词返回固定标签

报告里说明本地原型支持 Mock，生产环境可替换为真实 LLM API。

C-4 AI 合规检测

负责：

- 模拟或真实调用 AI 进行合规检测
- 返回 PASS / REVIEW / REJECT
- 保存检测结果

C-5 AI 流式续写 SSE

负责接口：

```
GET /api/ai/continue-writing/stream
```

实现内容：

- 接收 prompt
- 调用 Streaming API 或模拟逐句输出
- 使用 SSE 返回内容

C-6 Postman / Apifox 测试集

负责整理完整测试流程：

1. 注册
2. 登录
3. 创建草稿
4. 发布文章
5. 查看文章详情
6. 点赞
7. 评论
8. 查看热榜
9. 触发 AI 续写
10. 测试网关限流

交付物：

```
postman/AI-KnowledgeHub.postman_collection.json
```

C-7 演示视频

负责录制 3-5 分钟视频。

视频结构建议：

- 0:00 - 0:30 展示项目结构和 service 启动
- 0:30 - 1:00 展示 MySQL、Redis、RabbitMQ 启动状态
- 1:00 - 1:40 用户登录、创建文章、发布文章
- 1:40 - 2:20 展示 RabbitMQ 消费日志，说明标签提取和合规检测
- 2:20 - 3:00 展示阅读、点赞、评论后 Redis 热榜变化

3:00 - 3:40 展示 AI SSE 流式续写
3:40 - 4:30 展示网关限流效果和 Swagger 文档

C-8 报告负责章节

成员 C 负责报告中的：

- 5.6 RabbitMQ 异步广播处理
- 5.7 AI 流式创作辅助功能
- 6. 接口设计与测试
- 7. 本地运行与演示说明
- 8.3 为什么使用 RabbitMQ
- 8.5 为什么使用 SSE
- 9.5 MQ 生产环境方案
- 9.6 大模型调用生产级优化
- 8. 总结

十一、建议开发优先级

为了保证最后一定能交付，建议按下面顺序推进。

第一优先级：必须完成

这些是验收底线：

- 1. 至少 3 个微服务能启动
- 2. MySQL、Redis、RabbitMQ 能启动
- 3. 用户注册登录 + JWT
- 4. 文章发布、详情、点赞、评论
- 5. Redis ZSET 热榜 Top10
- 6. RabbitMQ 发布文章事件并被消费者收到
- 7. Swagger/OpenAPI 文档
- 8. Postman/Apifox Collection
- 9. 报告完整

第二优先级：高分关键

这些是拉开分数的地方：

1. **Gateway** 动态限流
2. **Fanout** 模式两个消费者独立消费同一篇文章
3. **AI** 流式续写 **SSE**
4. 报告中讲清楚生产环境高可用和高并发演进方案
5. 演示视频展示 **Redis**、**MQ**、**SSE**、限流完整链路

第三优先级：锦上添花

时间充足再做：

1. 文章详情缓存
2. 热度定时衰减
3. **Token** 黑名单
4. **MQ** 死信队列
5. **Docker** 一键启动全部服务
6. 简单前端测试页面
7. 管理员后台接口

十二、验收演示主线建议

你们最后演示时最好围绕一条完整链路讲，而不是零散展示接口。

推荐演示主线：

用户登录
-> 创建文章草稿
-> 发布文章
-> **RabbitMQ** 广播消息
-> **AI** 标签消费者处理
-> **AI** 合规消费者处理
-> 访问文章详情
-> 点赞评论
-> **Redis** 热榜变化
-> 调用 **AI** 流式续写
-> 测试网关限流

这条链路基本覆盖了老师要求的所有核心点。

十三、最终提交物清单

你们最后需要准备：

1. 作业报告 .docx / .pdf
2. 源代码工程
3. docker-compose.yml
4. README.md
5. Swagger/OpenAPI 地址或导出文件
6. Postman / Apifox Collection
7. 3-5 分钟演示视频
8. 小组分工说明

十四、我建议你们最终实现的最小可交付版本

为了避免项目做崩，建议最低做到这个程度：

```
gateway-service
user-service
article-service
ai-service
```

其中 ranking 功能可以放在 article-service 里，但报告中仍然可以把热榜模块独立描述。

更推荐做到：

```
gateway-service
user-service
article-service
ranking-service
ai-service
```

这样微服务架构更明显，报告也更好写。

十五、一句话总结

这个大作业的得分关键不是“文章 CRUD 写得多完整”，而是要让老师看到：

我们知道在高并发内容平台中，普通请求应该走网关，热点数据应该进 Redis，耗时 AI 任务应该进 MQ 异步处理，AI 生成应该流式返回，生产环境还要考虑服务多实例、缓存、消息可靠性和 LLM 调用限流。

按这个方向做，项目复杂度可控，验收展示也会比较有亮点。